

Taste Mender Search Benchmark

```
In [5]: # benchmark search script
import requests, time, statistics
import pandas as pd

BASE_URL = "http://127.0.0.1:8000/api/v1/search"
TIMEOUT = 10 # seconds

TASKS = [{
    "search_type": "track",
    "description": "3 char search (LIKE)",
    "queries": ["met", "dre", "dio", "dna", "the", "wal", "3fd"]
}, {
    "search_type": "track",
    "description": "one word search (FTS + TrigramWordDistance)",
    "queries": ["mozart", "adele", "hello", "rihanna", "umbrela", "beyond", "fsdfsdfsdf"]
}, {
    "search_type": "track",
    "description": "multi word search (FTS + TrigramDistance)",
    "queries": [
        "master of puppets", "smells like teen spirit", "ok compter", "lose yourself",
        "hotline bling", "the beatles", "jbdf saz bqrt"
    ]
}, {
    "search_type": "artist",
    "description": "3 char search (LIKE)",
    "queries": ["que", "abb", "emi", "nir", "dra", "met", "xqz"]
}, {
    "search_type": "artist",
    "description": "one word search (TrigramDistance)",
    "queries": ["queen", "abba", "eminem", "nirvana", "metalica", "drake", "qxyzmgfgd"]
}, {
    "search_type": "album",
    "description": "3 char search (LIKE)",
    "queries": ["thr", "abb", "rum", "rev", "hyb", "bac", "z9q"]
}, {
    "search_type": "album",
    "description": "one word search (TrigramDistance)",
    "queries": ["thriller", "back", "rumours", "revival", "hybrid", "abbey", "qxyzmgfgd"]
}]

def run_benchmark(run_label="run") -> pd.DataFrame:
    rows = []

    for task in TASKS:
        print(f"{task['search_type']} search with {task['description']}")
        response_times = []

        for i, query in enumerate(task["queries"], start=1):
            params = {
                "q": query,
                "type": task["search_type"],
                "limit": 50,
            }

            status = "error"
            status_code = None
            elapsed = None

            try:
                start = time.perf_counter()
```

```

response = requests.get(BASE_URL, params=params, timeout=TIMEOUT)
elapsed = time.perf_counter() - start
status_code = response.status_code

if response.status_code == 200:
    status = "ok"
    response_times.append(elapsed)
    print(f"    [{i}][{query}] OK {elapsed:.4f}s")
else:
    status = "http_error"
    print(f"    [{i}][{query}] FAIL {response.status_code}")

except requests.exceptions.RequestException as e:
    print(f"    [{i}][{query}] ERROR {e}")

rows.append({
    "run_label": run_label,
    "search_type": task["search_type"],
    "description": task["description"],
    "query": query,
    "status": status,
    "status_code": status_code,
    "elapsed_s": elapsed,
})

if response_times:
    print(f"Average: {statistics.mean(response_times):.4f}s")
    print(f"Median: {statistics.median(response_times):.4f}s")

return pd.DataFrame(rows)

```

```

In [6]: # regular run
df1 = run_benchmark(run_label="regular")
df1.head()

```

track search with 3 char search (LIKE)

[1][met] OK 0.1971s
[2][dre] OK 0.2863s
[3][dio] OK 0.2143s
[4][dna] OK 0.4944s
[5][the] OK 0.1566s
[6][wal] OK 0.1893s
[7][3fd] OK 5.9430s

Average: 1.0687s

Median: 0.2143s

track search with one word search (FTS + TrigramWordDistance)

[1][mozart] OK 0.4297s
[2][adele] OK 0.1701s
[3][hello] OK 0.1556s
[4][rihanna] OK 0.1384s
[5][umbrela] OK 1.5105s
[6][beyond] OK 0.2010s
[7][fsdfsdfsdf] OK 1.3172s

Average: 0.5604s

Median: 0.2010s

track search with multi word search (FTS + TrigramDistance)

[1][master of puppets] OK 0.1658s
[2][smells like teen spirit] OK 0.1519s
[3][ok compter] OK 1.6613s
[4][lose yourself] OK 0.1872s
[5][hotline bling] OK 1.7335s
[6][the beatles] OK 0.2192s
[7][jbdf saz bqrt] OK 1.5403s

Average: 0.8084s

Median: 0.2192s

artist search with 3 char search (LIKE)

[1][que] OK 0.0641s
[2][abb] OK 0.0929s
[3][emi] OK 0.0701s
[4][nir] OK 0.1730s
[5][dra] OK 0.0593s
[6][met] OK 0.0766s
[7][xqz] OK 0.2862s

Average: 0.1174s

Median: 0.0766s

artist search with one word search (TrigramDistance)

[1][queen] OK 0.1085s
[2][abba] OK 0.0740s
[3][eminem] OK 0.0984s
[4][nirvana] OK 0.0872s
[5][metalica] OK 0.0922s
[6][drake] OK 0.0855s
[7][qxyzmgfgd] OK 0.0812s

Average: 0.0896s

Median: 0.0872s

album search with 3 char search (LIKE)

[1][thr] OK 0.1052s
[2][abb] OK 0.1467s
[3][rum] OK 0.0817s
[4][rev] OK 0.1150s
[5][hyb] OK 0.3192s
[6][bac] OK 0.0791s
[7][z9q] OK 0.7548s

Average: 0.2288s

Median: 0.1150s

album search with one word search (TrigramDistance)

[1][thriller] OK 0.2553s
[2][back] OK 0.1861s
[3][rumours] OK 0.1651s
[4][revival] OK 0.1946s
[5][hybrid] OK 0.2006s

```
[6][abbey] OK 0.1769s
[7][qxyzmgfgd] OK 0.1587s
Average: 0.1910s
Median: 0.1861s
```

Out[6]:

	run_label	search_type	description	query	status	status_code	elapsed_s
0	regular	track	3 char search (LIKE)	met	ok	200	0.197127
1	regular	track	3 char search (LIKE)	dre	ok	200	0.286311
2	regular	track	3 char search (LIKE)	dio	ok	200	0.214273
3	regular	track	3 char search (LIKE)	dna	ok	200	0.494355
4	regular	track	3 char search (LIKE)	the	ok	200	0.156628

```
In [7]: # run with trigram always enabled
df2 = run_benchmark(run_label="trigram_always_on")
df2.head()
```

track search with 3 char search (LIKE)

[1][met] OK 0.2483s
[2][dre] OK 0.1601s
[3][dio] OK 0.1670s
[4][dna] OK 0.1950s
[5][the] OK 0.7983s
[6][wal] OK 1.1039s
[7][3fd] OK 1.1048s

Average: 0.5396s

Median: 0.2483s

track search with one word search (FTS + TrigramWordDistance)

[1][mozart] OK 0.3266s
[2][adele] OK 0.1734s
[3][hello] OK 0.1588s
[4][rihanna] OK 0.1336s
[5][umbrela] OK 1.4906s
[6][beyond] OK 0.1973s
[7][fsdfsdfsdf] OK 1.3049s

Average: 0.5407s

Median: 0.1973s

track search with multi word search (FTS + TrigramDistance)

[1][master of puppets] OK 0.2711s
[2][smells like teen spirit] OK 0.1413s
[3][ok compter] OK 1.6376s
[4][lose yourself] OK 0.1632s
[5][hotline bling] OK 1.7315s
[6][the beatles] OK 0.1976s
[7][jbdf saz bqrt] OK 1.5195s

Average: 0.8088s

Median: 0.2711s

artist search with 3 char search (LIKE)

[1][que] OK 0.1328s
[2][abb] OK 0.0963s
[3][emi] OK 0.1231s
[4][nir] OK 0.0914s
[5][dra] OK 0.1828s
[6][met] OK 0.0912s
[7][xqz] OK 0.0723s

Average: 0.1129s

Median: 0.0963s

artist search with one word search (TrigramDistance)

[1][queen] OK 0.1088s
[2][abba] OK 0.0969s
[3][eminem] OK 0.0983s
[4][nirvana] OK 0.1261s
[5][metalica] OK 0.0928s
[6][drake] OK 0.1171s
[7][qxyzmgfgd] OK 0.1023s

Average: 0.1061s

Median: 0.1023s

album search with 3 char search (LIKE)

[1][thr] OK 0.2494s
[2][abb] OK 0.1314s
[3][rum] OK 0.1838s
[4][rev] OK 0.1717s
[5][hyb] OK 0.1725s
[6][bac] OK 0.1358s
[7][z9q] OK 0.1564s

Average: 0.1716s

Median: 0.1717s

album search with one word search (TrigramDistance)

[1][thriller] OK 0.1794s
[2][back] OK 0.1375s
[3][rumours] OK 0.1742s
[4][revival] OK 0.1730s
[5][hybrid] OK 0.1730s

```
[6][abbey] OK 0.2659s
[7][qxyzmgfgd] OK 0.1692s
Average: 0.1817s
Median: 0.1730s
```

Out[7]:

	run_label	search_type	description	query	status	status_code	elapsed_s
0	trigram_always_on	track	3 char search (LIKE)	met	ok	200	0.248254
1	trigram_always_on	track	3 char search (LIKE)	dre	ok	200	0.160139
2	trigram_always_on	track	3 char search (LIKE)	dio	ok	200	0.166974
3	trigram_always_on	track	3 char search (LIKE)	dna	ok	200	0.195008
4	trigram_always_on	track	3 char search (LIKE)	the	ok	200	0.798340

```
In [8]: # compare multiple runs
all_runs = pd.concat([df1, df2], ignore_index=True)

comparison = (
    all_runs[all_runs["status"] == "ok"]
    .groupby(["run_label", "search_type", "description"], as_index=False)
    .agg(
        requests=("elapsed_s", "size"),
        mean_elapsed_s=("elapsed_s", "mean"),
        median_elapsed_s=("elapsed_s", "median"),
        p95_elapsed_s=("elapsed_s", lambda s: s.quantile(0.95)),
    )
    .sort_values(["search_type", "description", "run_label"])
    .reset_index(drop=True)
)

comparison
```

Out[8]:

	run_label	search_type	description	requests	mean_elapsed_s	median_elapsed_s
0	regular	album	3 char search (LIKE)	7	0.228805	0.114984
1	trigram_always_on	album	3 char search (LIKE)	7	0.171570	0.171665
2	regular	album	one word search (TrigramDistance)	7	0.191036	0.186112
3	trigram_always_on	album	one word search (TrigramDistance)	7	0.181749	0.173041
4	regular	artist	3 char search (LIKE)	7	0.117440	0.076553
5	trigram_always_on	artist	3 char search (LIKE)	7	0.112850	0.096295
6	regular	artist	one word search (TrigramDistance)	7	0.089570	0.087158
7	trigram_always_on	artist	one word search (TrigramDistance)	7	0.106056	0.102282
8	regular	track	3 char search (LIKE)	7	1.068712	0.214273
9	trigram_always_on	track	3 char search (LIKE)	7	0.539639	0.248254
10	regular	track	multi word search (FTS + TrigramDistance)	7	0.808433	0.219194
11	trigram_always_on	track	multi word search (FTS + TrigramDistance)	7	0.808823	0.271112
12	regular	track	one word search (FTS + TrigramWordDistance)	7	0.560364	0.201017
13	trigram_always_on	track	one word search (FTS + TrigramWordDistance)	7	0.540723	0.197265



In [9]:

```
# concise comparison summary
ok = all_runs[all_runs["status"] == "ok"].copy()

summary = (
    ok.groupby(["run_label"], as_index=False)
    .agg(
        requests=("elapsed_s", "size"),
        mean_elapsed_s=("elapsed_s", "mean"),
        median_elapsed_s=("elapsed_s", "median"),
        p95_elapsed_s=("elapsed_s", lambda s: s.quantile(0.95)),
    )
    .sort_values("run_label")
    .reset_index(drop=True)
)

by_task = (
    ok.groupby(["run_label", "search_type", "description"], as_index=False)
    .agg(
        mean_elapsed_s=("elapsed_s", "mean"),
        median_elapsed_s=("elapsed_s", "median"),
        p95_elapsed_s=("elapsed_s", lambda s: s.quantile(0.95)),
    )
    .sort_values(["search_type", "description", "run_label"])
)
```

```

pivot = by_task.pivot(index=["search_type", "description"], columns="run_label", values="mean_
if set(["regular", "trigram_always_on"]).issubset(pivot.columns):
    pivot["delta_ms"] = (pivot["trigram_always_on"] - pivot["regular"]) * 1000
    pivot["pct_change"] = ((pivot["trigram_always_on"] / pivot["regular"]) - 1.0) * 100
    pivot = pivot.sort_values("pct_change")

print("Overall summary by run:")
print(summary.to_string(index=False))
print("\nPer-task mean comparison:")
print(pivot.to_string())

```

Overall summary by run:

	run_label	requests	mean_elapsed_s	median_elapsed_s	p95_elapsed_s
	regular	49	0.437766	0.170077	1.612879
	trigram_always_on	49	0.351630	0.171665	1.507928

Per-task mean comparison:

run_label		regular	trigram_always_on	delta_
ms	pct_change			
search_type	description			
track	3 char search (LIKE)	1.068712	0.539639	-529.0721
71	-49.505606			
album	3 char search (LIKE)	0.228805	0.171570	-57.2352
00	-25.014854			
	one word search (TrigramDistance)	0.191036	0.181749	-9.2862
57	-4.861007			
artist	3 char search (LIKE)	0.117440	0.112850	-4.5895
43	-3.908002			
track	one word search (FTS + TrigramWordDistance)	0.560364	0.540723	-19.6415
71	-3.505142			
	multi word search (FTS + TrigramDistance)	0.808433	0.808823	0.3897
00	0.048204			
artist	one word search (TrigramDistance)	0.089570	0.106056	16.4857
57	18.405448			